Ant Colony Optimization (ACO) for the Traveling Salesman Problem (TSP)

Tristan Froese
Dept. of Computer Science, Brock University
St. Catharines, Canada
ka23he@brocku.ca
7854607

Devaan Zastrow
Dept. of Computer Science, Brock University
St. Catharines, Canada
dz19xr@brocku.ca
6932842

***Abstract*—The Traveling Salesman Problem (TSP) is a classic optimization problem in which a salesman must determine the shortest possible route that visits a set of cities exactly once and returns to the starting city. As the number of cities increases, the number of possible routes increases factorially, making exact solutions computationally expensive and sometimes impractical for larger problems. Traditional deterministic algorithms struggle with scalability, motivating the exploration of alternative heuristic and bio inspired optimization methods. Swarm intelligence techniques like Ant Colony Optimization (ACO), mimic the cooperative behavior of ants to discover solutions through decentralized decision making and indirect communication. However, practically implementing and evaluating such systems remains a challenge, particularly in balancing exploration and exploitation to avoid local optima. This project addresses the need for a practical implementation of Ant Colony Optimization using TSP as a benchmark problem**

I. Introduction

The Traveling Salesman Problem (TSP) remains one of the most widely studied problems in combinatorial optimization and theoretical computer science. Formally, the problem is defined as finding the shortest possible route through a set of n cities such that each city is visited exactly once and the route returns to the original starting city. The cities can be represented as nodes in a weighted graph, where the edges represent distances between cities. The objective is therefore to determine the Hamiltonian cycle with the minimum total cost [1].

The difficulty of the TSP arises from the rapid growth of its solution space. For a fully connected graph of n cities, the number of unique tours is $(n-1)!/2$; This factorial growth means that even moderately sized instances can produce an enormous number of possible routes. For example, while a problem containing 10 cities has roughly 180,000 possible paths, a problem with 20 cities already exceeds 10^{18} potential routes. Because of this explosive growth, exhaustive search methods become completely infeasible for any kind of larger problem.

As a consequence, the TSP is classified as an NP-hard problem [2]. While exact algorithms such as dynamic programming or branch-and-bound methods can guarantee optimal solutions for smaller instances, their computational cost increases dramatically as problem size grows. This has led researchers to explore heuristic approaches that can produce near-optimal solutions with significantly lower computational effort [3].

One particularly successful class of heuristic algorithms is based on swarm intelligence. Swarm intelligence refers to the collective problem-solving behavior that emerges from groups of relatively simple agents interacting locally with one another and with their environment. Unlike centralized optimization methods, swarm-based algorithms rely on distributed decision making and indirect communication to coordinate search behavior.

Among these, Ant Colony Optimization (ACO) is a leading metaheuristic modeled after the foraging behavior of biological ants [4] [5]. In nature, ants deposit chemical markers known as pheromones to communicate the quality of a path to their peers through a process known as stigmergy. As ants repeatedly travel along more efficient paths, these routes accumulate more pheromones, increasing the likelihood that future ants will follow them. Conversely, less efficient paths gradually lose pheromones through evaporation, reducing their attractiveness over time.

In this context, artificial ants construct solutions probabilistically, biased by both pheromone intensity (history) and heuristic desirability (distance). However, the effectiveness of ACO is highly sensitive to the delicate balance between exploration and exploitation. If the pheromone influence is too strong, the colony may converge prematurely on a local optimum. On the other hand, if the heuristic is weighted too heavily, it can lead to slow convergence and suboptimal path lengths.

This project implements an ACO framework to evaluate performance against standard benchmarks from the TSP LIB library. By manipulating key parameters such as the pheromone evaporation rate (ρ) and the relative importance of history and distance (α, β), this study aims to quantify the adaptability of swarm intelligence in solving complex logistic based problems.

II. Methodology

The proposed solution utilizes the ACO metaheuristic to approximate optimal tours for various instances of the TSP. The algorithm simulates the stigmergic communication observed in natural ant colonies, where simple agents collectively produce

intelligent behavior through repeated interactions with their environment.

In the context of the TSP, cities are represented as nodes in a graph, while edges correspond to the distances between cities. Artificial ants traverse this graph to construct candidate tours. During this process, ants rely on both historical information encoded in pheromone trails and heuristic information derived from the problem structure. Over successive iterations, pheromone reinforcement allows the colony to gradually concentrate its search around promising solutions.

A. Solution Construction

During each iteration of the algorithm, a population of m ants is initialized. Each ant begins at a starting city and incrementally constructs a complete tour by repeatedly selecting the next city to visit. At any given step, an ant located at city i chooses its next destination j from the set of cities that it has not yet visited.

The choice of the next city is not deterministic; instead, it follows a probabilistic transition rule designed to balance exploration and exploitation. The probability that an ant moves from city i to city j is defined as:

$$P_{ij}^k = \frac{[\tau_{ij}]^\alpha \cdot [\eta_{ij}]^\beta}{\sum_{l \in J_i^k} [\tau_{il}]^\alpha \cdot [\eta_{il}]^\beta}$$

where τ_{ij} represents the pheromone concentration on the edge (i,j) and η_{ij} represents the heuristic desirability, calculated as the inverse of the distance $1/d_{ij}$ in order to encourages ants to favor shorter edges when constructing their tours. The parameters α and β control the relative influence of the pheromone trail and the visibility, respectively. When α is large, ants rely more heavily on accumulated pheromone information, which intensifies the search around previously successful paths. Similarly, larger values of β increase the influence of the distance heuristic, encouraging ants to prioritize shorter edges during path construction.

B. Pheromone Update Rule

Once all ants have completed their tours during an iteration, the pheromone levels on the graph edges are updated to reflect the quality of the solutions discovered. This update process consists of two main components: pheromone evaporation and pheromone deposition.

First, pheromone evaporation is applied uniformly across all edges. This is modeled by multiplying the existing pheromone levels by a factor of $(1 - \rho)$ where $\rho \in [0, 1]$ represents the evaporation rate. By gradually reducing pheromone levels over time, the algorithm maintains diversity in the search process and reduces the risk of premature convergence.

After evaporation, ants deposit additional pheromone along the edges they traversed in their tours. Each ant k deposits a quantity of pheromone $\Delta\tau_{ij}^k$ along the edges it traversed:

$$\tau_{ij} \leftarrow (1 - \rho)\tau_{ij} + \sum_{k=1}^{m} \Delta\tau_{ij}^k$$

The amount of pheromone deposited is inversely proportional to the total distance L_k of the ant's tour, such that $\Delta\tau_{ij}^k = Q/L_k$, where Q is a constant. Over successive iterations, this feedback mechanism allows the colony to progressively concentrate pheromone on the most promising routes, guiding the search toward near-optimal solutions.

C. Complexity Analysis

The computational complexity of the implemented ACO algorithm is primarily driven by the solution construction. For each iteration N, every ant m must visit n cities. At each city the ant must evaluate the transition probability for all remaining unvisited cities

*1) **Solution Construction**:* In the worst case, the construction of a single tour requires evaluating transition probabilities for up to n candidate cities at each of the n steps in the tour, scaling it at $O(n^2)$. With m ants, the complexity of the solution construction phase becomes $O(m \cdot n^2)$.

*2) **Pheromone Update**:* Updating the pheromone matrix involves iterating over all edges in the graph to apply evaporation and add new pheromone deposits. Since a complete graph of n cities contains on the order of n^2 edges, the pheromone update step also requires $O(n^2)$ time.

*3) **Total Complexity**:* Across N iterations, the total time complexity can be approximated as

$$O(N \cdot m \cdot n^2)$$

While this is significantly more efficient than the $(n - 1)!$ complexity of brute-force search, it still grows rapidly with problem size, and highlights why larger instances like *gr120* require significantly more computation time than smaller instances like *berlin52*.

III. Experimental Setup

To evaluate the performance of the proposed ACO algorithm, a series of controlled experiments were conducted. The goal was to determine how variations in the heuristic parameters α and β, along with the evaporation rate ρ, affects the convergence behavior and final tour length.

A. Hardware and Software Environment

The algorithm was implemented in Python 3.11, using the NumPy library for matrix operations and Matplotlib for data visualization. All simulations were performed on an AMD Ryzen 5 3600 6-Core Processor with 32GB of memory

B. Benchmark Datasets

The performance was benchmarked using examples from the TSPLIB library [6], which provides over 100 coordinate-based city locations along with known optimal distances for each, allowing for algorithms to be tested and compared. The following instances were selected for closer examination based on their varying levels of complexity:

- berlin52: A 52-city problem representing locations in Berlin.
- eil51: A 51-city synthetic dataset
- gr120: A 120-city problem representing locations in Greece

C. Parameter Configuration

The baseline parameters used for the initial runs on the TSPLIB benchmark problems are summarized in Table I. These values were selected on the basis of standard ACO conventions to ensure a balance between exploration and exploitation.

TABLE I: Initial ACO Parameter Settings

Parameter	Symbol	Value
Number of Ants	m	50
Pheromone Importance	α	1.0
Distance Importance	β	5.0
Evaporation Rate	ρ	0.5
Pheromone Constant	Q	100
Maximum Iteration	N	150

Note: Parameters listed here are for the performance benchmarks in Section IV; visualization parameters are detailed in Section IV-E.

D. Performance Metrics

These metrics were used to assess the algorithm's success:

- **Best Tour Length:** The shortest path distance identified by the algorithm across all iterations. This represents the absolute best solution found during the runtime.
- **Mean Relative Error (MRE):** To determine the overall accuracy and stability of the solver across multiple trials, the error relative to the known TSPLIB optimal solution was calculated using the mean results of 10 independent runs:

$$MRE = \left(\frac{\text{Mean Tour Length} - \text{Optimal Length}}{\text{Optimal Length}} \right) \tag{1}$$

- **Convergence Rate:** The number of iterations required before the "Early Stopping" mechanism was triggered, or before the pheromone levels stabilized.
- **Computation Time:** The total CPU time (in seconds) required to complete the search, measured using Python's `time` module.

E. Procedure

Each instance was repeated for 10 independent trials to account for the stochastic nature of the algorithm. To ensure data integrity, the `load_tour` function was used to validate the results against standard solutions, with a logic fix applied to the early stopping mechanism to prevent premature termination during high-error phases.

IV. Results and Discussion

A. Algorithm Performance

The performance of the algorithm was evaluated across multiple TSPLIB benchmarks. Table II summarizes the best tour lengths achieved compared to the known optimal solutions for our selected focus instances. Figures 2 to 4 display the results of the convergence of these instances, as obtained using the initial parameter set from Table I, averaged over 10 independent runs; Figures 5 to 7 showcase the best paths found during these runs. Finally, Table V displays some initial results for a larger sample of instances.

TABLE II: Comprehensive Performance Analysis of the ACO Algorithm (10 Trials)

	Solution Quality					Efficiency	
Instance	Best	Avg. Len.	Std. Dev.	Optimal	MRE(%)	Time (s)	Conv. Iter.
berlin52.tsp	7658.00	7786.40	99.08	7542.00	3.24%	8.98	38.40
eil51.tsp	443.00	453.70	5.90	426.00	6.50%	8.24	33.60
gr120.tsp	1743.00	1794.50	25.60	1667.00	7.65%	40.54	46.10

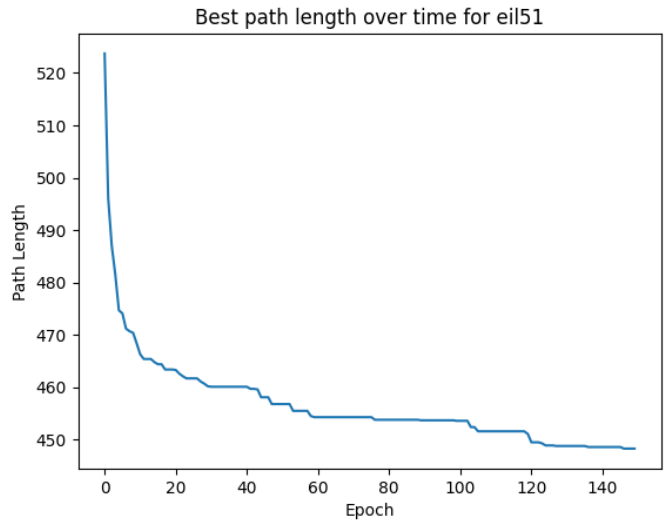


Fig. 1: eil51's best path length over time, averaged over 10 iterations

B. Analysis of Convergence

As shown in the results, the algorithm performed exceptionally well on smaller instances like `berlin52`. However, as the datasets became larger, such as with `gr120`, a trending

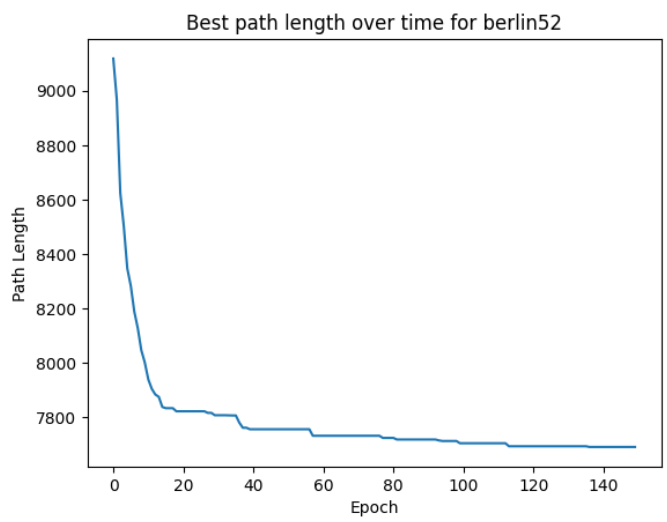


Fig. 2: berlin52's best path length over time, averaged over 10 iterations

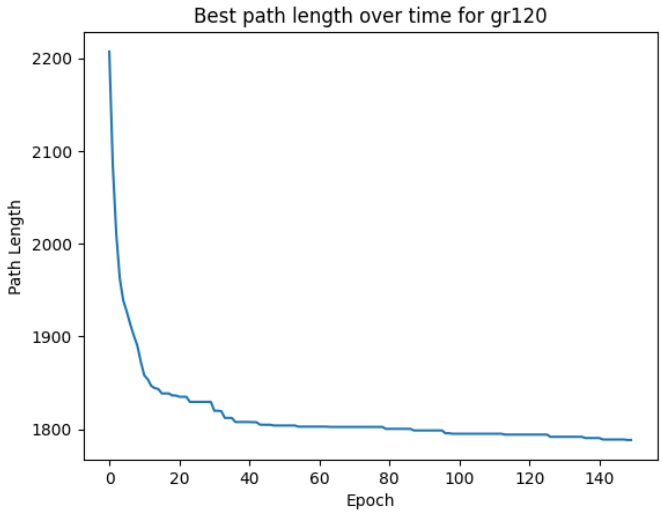


Fig. 3: gr120's best path length over time, averaged over 10 iterations

increase in error was observed. This is most likely due to the factorial increase in the search space, which may require a higher pheromone evaporation rate or a more aggressive early stopping threshold to navigate effectively.

C. Impact of Early Stopping

The implementation of the logic fix in the early stopping mechanism significantly reduced computation time without sacrificing accuracy. By monitoring the stabilization of the best tour length over 30 iterations, the algorithm was allowed to avoid unnecessary cycles once the pheromone trails had effectively converged on a local optimum.

D. Hyperparameter Sensitivity Analysis

To evaluate the performance of the implemented ACO algorithm, a series of sensitivity tests were conducted on the gr120 instance. The objective was to observe how deviations from the baseline parameters ($m = 50$, $\alpha = 1.0, \beta = 5.0, \rho = 0.5$) influence the balance between exploration (searching new paths) and exploitation (refining known paths).

TABLE III: Sensitivity of Solution Quality to Parameter Variations (Instance: gr120)

Parameter	Value	Avg. Len	StdDev	MRE (%)
Baseline	$\alpha = 1.0, \beta = 5.0, \rho = 0.5$	1794.50	25.60	7.65%
Pheromone (α)	Low (0.1)	1988.50	27.24	19.29%
	High (5.0)	1835.40	47.05	10.10%
Heuristic (β)	Low (1.0)	2127.40	41.86	27.62%
	High (10.0)	1812.40	8.32	8.72%
Evaporation (ρ)	Low (0.1)	1819.60	25.40	9.15%
	High (0.9)	1802.60	25.75	8.13%

1) Influence of Pheromone Weight (α): The α parameter regulates the colony's "social memory".

- **Low α (0.1):** When the influence of the pheromone is minimized, the ants rely almost exclusively on local distance heuristics. This resulted in a higher standard deviation, as the lack of collective guidance led to inconsistent path discovery across trials.
- **High α (5.0):** High α led to unstable reinforcement. The significantly higher StdDev suggests that the colony prematurely committed to various local optima in different trials, preventing consistent results.

2) Influence of Heuristic Desirability (β): β represents the "visibility" of individual agents.

- **Low β (1.0):** With negligible heuristic guidance, the initial iterations resembled a random walk. The convergence rate was significantly delayed, as the ants struggled to identify a structural baseline for the tour.
- **High β (10.0):** Interestingly, High β provided the highest stability. This suggests that for the gr120 instance, a strong greedy bias significantly aids in narrowing the search space to high-quality local edges.

3) Pheromone Evaporation (ρ): Evaporation acts as a mechanism to prevent the infinite accumulation of pheromones, allowing the colony to "forget" sub-optimal pathways.

- **Low Evaporation (0.1):** At this level, 90% of the pheromone is retained each epoch. This created an excessively strong "collective memory" that made it difficult for the colony to abandon early sub-optimal paths. Consequently, this produced the highest error among the tested ρ values (9.15% MRE), as the ants remained influenced by poor initial tours.
- **High Evaporation (0.9):** In this instance, only 10% of the pheromone persists. This forces the colony to constantly reinforce the quality of a path through immediate and heavy reinforcement. Interestingly, for the gr120 instance, this resulted in an 8.13% MRE, which outperformed the low evaporation setting. This suggests that for more complex maps, a "short-term memory" that prioritizes recent, successful discoveries can be more effective than a long-term memory that risks stagnation.

4) Summary of Observations: The experimental analysis of the gr120 instance reveals critical insights into the balance between collective intelligence and individual greed within the Ant Colony Optimization framework. The following observations summarize the key findings:

- The results indicate that Heuristic Desirability (β) is the primary determinant of stability. Increasing β to 10.0 provided very consistent results in all trials. This suggests that a strong heuristic bias plays a vital role in spaces, effectively pruning low-quality edges before pheromone trails have even been established.
- While pheromones allow the colony to learn, a high Pheromone Weight ($\alpha = 5.0$) introduced significant instability. Rather than converging on a single global optimum, the colony became hypersensitive to early, random discoveries. This led the algorithm to converge on entirely different local optima, as evidenced by the high Standard Deviation (47.05).

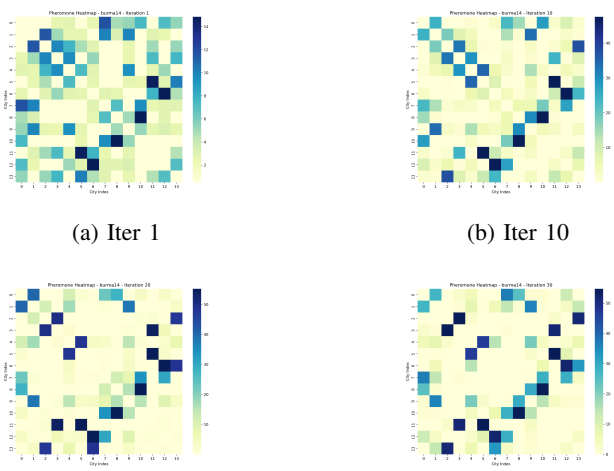
- Perhaps the most counter-intuitive finding was the success of High Evaporation ($\rho = 0.9$). By retaining only 10% of pheromones per epoch, the colony was forced to continuously validate path quality. This "short-term memory" prevented the algorithm from premature convergence on sub-optimal initial paths, allowing it to outperform the low evaporation setting by nearly 1% MRE.

E. Visual Analysis of Pheromone placement (Heatmaps)

To better illustrate the dynamic shift from exploration to exploitation, the parameters for the heatmaps were adjusted to allow for a more gradual convergence. By utilizing a smaller ant population and β value on the *burma14* instance, the incremental reinforcement of pheromone trails can be more clearly observed.

TABLE IV: Parameter Configuration for Pheromone Visualization (*burma14*)

Parameter	Symbol	Value
Number of Ants	m	10
Pheromone Importance	α	1.0
Heuristic Importance	β	2.0
Evaporation Rate	ρ	0.5



(a) Iter 1 (b) Iter 10

(c) Iter 20 (d) Iter 30

Fig. 4: Pheromone evolution over 30 iterations

*1) **Iteration 1 - High Entropy Exploration** :* In the initial epoch, the pheromone distribution is nearly uniform, characterized by a high-entropy state. At this stage, the ants' movements are primarily driven by the heuristic desirability (β) rather than established trails. The scattered intensity across the heatmap indicates that the colony is broadly exploring the search space to identify potential candidate edges.

*2) **Iteration 10 - Positive Feedback and Reinforcement** :* By Iteration 10, we see agents prioritizing existing pheromones. Specific city pairs begin to show significantly higher intensity. This indicates that ants completing shorter tours have deposited more pheromone (Q/L), effectively "recruiting" subsequent ants to those paths. The maximum pheromone concentration has increased, indicating a transition from global exploration to local exploitation.

*3) **Iteration 20 - Path Stabilization**:* The matrix begins to simplify as the evaporation mechanism (ρ) successfully prunes sub-optimal trails. The "noise" from the initial random walks is being forgotten by the system, allowing a distinct structure to emerge. The symmetry of the matrix becomes more pronounced, reflecting the bidirectional nature of the travel costs in the symmetric TSP.

*4) **Iteration 30 - Convergence and Consensus** :* At Iteration 30, the system has reached a stable equilibrium. The heatmap is now highly sparse, with intensity concentrated on a very small subset of edges that form a single TSP cycle. The dark blue cells correspond to the high-probability transitions that define the final solution. This clear, low-entropy matrix confirms that the parameters ($\alpha = 1.0, \beta = 5.0$) allowed for a healthy exploration phase followed by a decisive convergence on a high-quality solution.

F. Closing Remarks

Overall, this project was able to highlight the effectiveness and flexibility of swarm intelligence approaches for more complex optimization problems, and emphasized the importance of parameter tuning and finding the balance between exploration and exploitation to achieve strong performance.

Consistently achieving below 15% MRE on a variety of differently-sized datasets without even delving into fine-tuning the network parameters seems quite a solid achievement, and showcases the large amount of raw potential that ACO has for more efficiently solving the TSP.

References

[1] Applegate, D. L.; Bixby, R. M.; Chvátal, V.; Cook, W. J. (2006), The Traveling Salesman Problem, Princeton University Press, ISBN 978-0-691-12993-8.

[2] Knuth, Donald (1974). "Postscript about NP-hard problems". ACM SIGACT News. 6 (2): 15–16. doi:10.1145/1008304.1008305. S2CID 46480926

[3] Alanzi, E., Menai, M.E.B. Solving the traveling salesman problem with machine learning: a review of recent advances and challenges. Artif Intell Rev 58, 267 (2025). https://doi.org/10.1007/s10462-025-11267-x

[4] M. Dorigo, M. Birattari and T. Stutzle, "Ant colony optimization," in IEEE Computational Intelligence Magazine, vol. 1, no. 4, pp. 28-39, Nov. 2006, doi: 10.1109/MCI.2006.329691.

[5] López-Ibáñez, M., Stützle, T., Dorigo, M. (2025). Ant Colony Optimization: A Component-Wise Overview. In: Martí, R., Pardalos, P.M., Resende, M.G. (eds) Handbook of Heuristics. Springer, Cham. https://doi.org/10.1007/978-3-319-07153-4_21-2

[6] G. Reinelt, "TSPLIB - A Library of Sample Instances for the TSP," GitHub Repository, 2019. [Online]. Available: https://github.com/shredderzwj/TSPLIB

[7] B. Thomas, "Traveling Salesman Problem with Time Windows - Benchmark Data Sets," University of Iowa, Sept. 2010. [Online]. Available: http://myweb.uiowa.edu/bthoa/TSPTWBenchmarkDataSets.htm

[8] L. Strika, "Ant Colony Optimization for the Traveling Salesman Problem," Sept. 2022. [Online]. Available: https://strikingloo.github.io/ant-colony-optimization-tsp

[9] W. M. Eido and I. M. Ibrahim, "Ant Colony Optimization (ACO) for Traveling Salesman Problem: A Review," *Asian J. Res. Comput. Sci.*, vol. 18, no. 2, pp. 20–45, Jan. 2025. doi: 10.9734/ajrcos/2025/v18i2559.

[10] Y. Wang and Z. Han, "Ant Colony Optimization for Traveling Salesman Problem Based on Parameters Optimization," *Appl. Soft Comput.*, vol. 107, p. 107439, Aug. 2021. doi: 10.1016/j.asoc.2021.107439.

[11] V. Murugananthan et al., "Traveling Salesman Problem with Ant Colony Optimization," in *Proc. 2nd Int. Conf. Electr. Comput. Commun. Anal. (ICECAA)*, July 2023. doi: 10.1109/icecaa58104.2023.10212262.

[12] D. T. Tran et al., "NeuFACO: Neural Focused Ant Colony Optimization for Traveling Salesman Problem," *arXiv preprint arXiv:2509.16938*, Sept. 2025.

[13] B. V. Raghavendra, "Solving Traveling Salesman Problem using Ant Colony Optimization Algorithm," *J. Comput. Sci. Syst. Biol.*, Apr. 2016. [Online]. Available: https://www.hilarispublisher.com/open-access/solving-traveling-salesmen-problem-using-ant-colony-optimizationalgorithm-2168-9679-1000260.pdf

TABLE V: Preliminary Results (10 ants, 200 epochs, α=1.0, β=2.0, ρ=0.9, averaged over 10 runs)

File Name	Average Length	Average Time (sec)	Best Length	Optimal Length	MRE (%)	Std Dev of Lengths
ulysses16	73.00	1.50	73.00	73.00	0.00	0.00
ulysses22	74.00	2.01	74.00	74.00	0.00	0.00
bayg29	9373.40	3.06	9220.00	9073.00	3.31	99.80
bays29	9448.00	3.04	9369.00	9290.00	1.70	70.62
att48	36077.00	6.31	34737.00	33522.00	7.62	692.48
eil51	453.70	8.44	443.00	426.00	6.50	5.90
berlin52	7786.40	6.58	7658.00	7542.00	3.24	99.08
st70	744.20	11.46	727.00	675.00	10.25	12.30
eil76	577.40	13.27	562.00	538.00	7.32	9.52
pr76	120637.60	17.39	119413.00	108159.00	11.54	1828.89
gr96	565.00	19.58	552.00	514.00	9.92	12.98
kroA100	23946.20	21.59	22796.00	21282.00	12.52	692.88
kroC100	22358.80	20.92	21633.00	20749.00	7.76	696.73
kroD100	23601.40	20.87	22923.00	21294.00	10.84	354.41
rd100	9076.20	21.01	8400.00	7910.00	14.74	374.75
eil101	733.00	21.07	707.00	629.00	16.53	22.55
lin105	15630.80	25.84	14970.00	14379.00	8.71	529.48
gr120	1794.50	37.71	1743.00	1667.00	7.65	25.60
ch130	6782.40	32.40	6519.00	6110.00	11.00	136.46
ch150	7277.20	42.09	7169.00	6528.00	11.48	92.12
brg180	2092.00	57.33	2020.00	1950.00	7.28	54.92
gr202	578.40	70.45	562.00	547.00	5.74	14.95
tsp225	4848.80	85.32	4634.00	3916.00	23.82	195.57
a280	3565.00	127.71	3441.00	2579.00	38.23	78.77
pcb442	74208.80	306.10	71292.00	50778.00	46.14	2107.22
pa561	22275.60	500.78	21457.00	19311.00	15.35	557.13

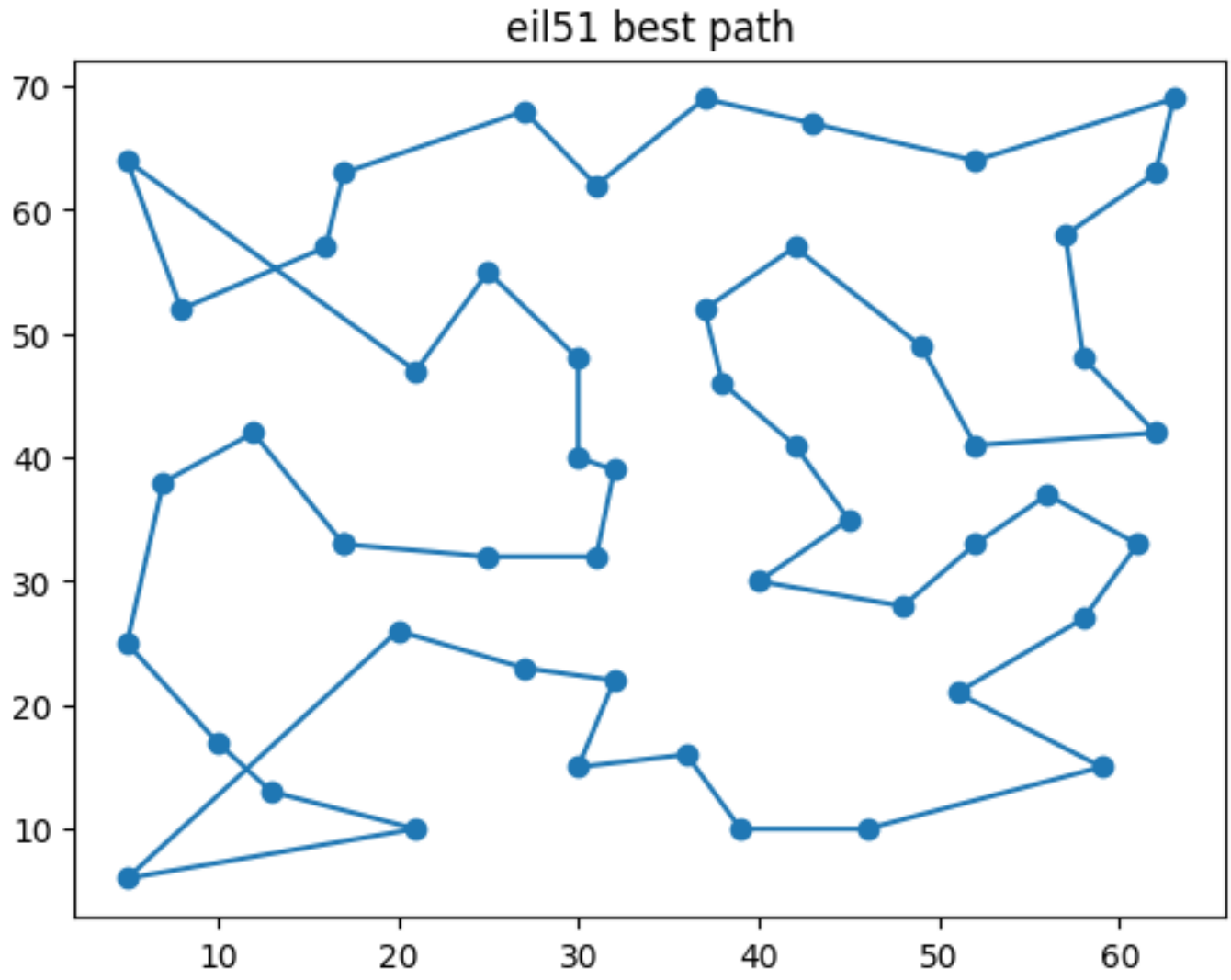


Fig. 5: eil51's best route from among 10 iterations

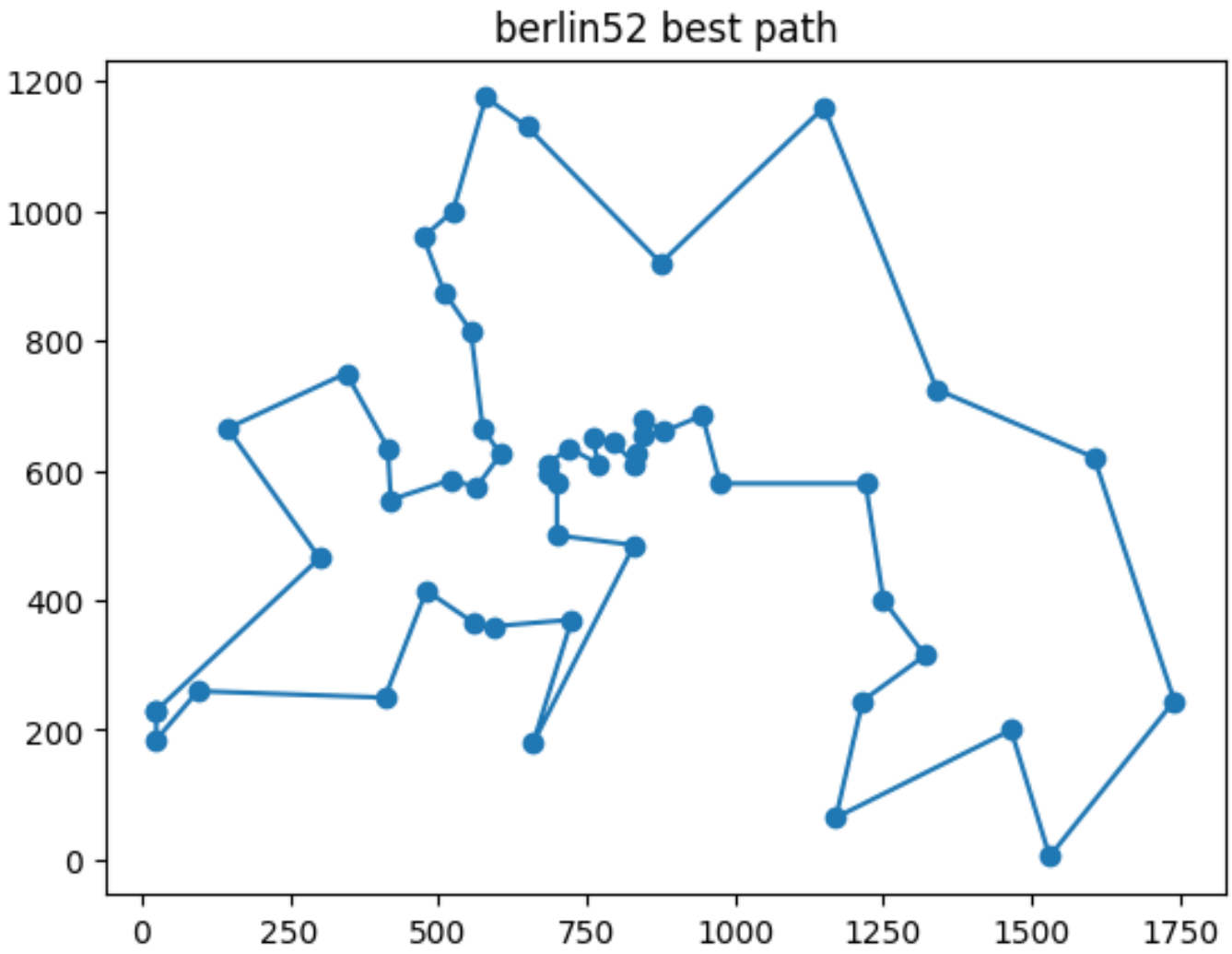


Fig. 6: berlin52's best route from among 10 iterations

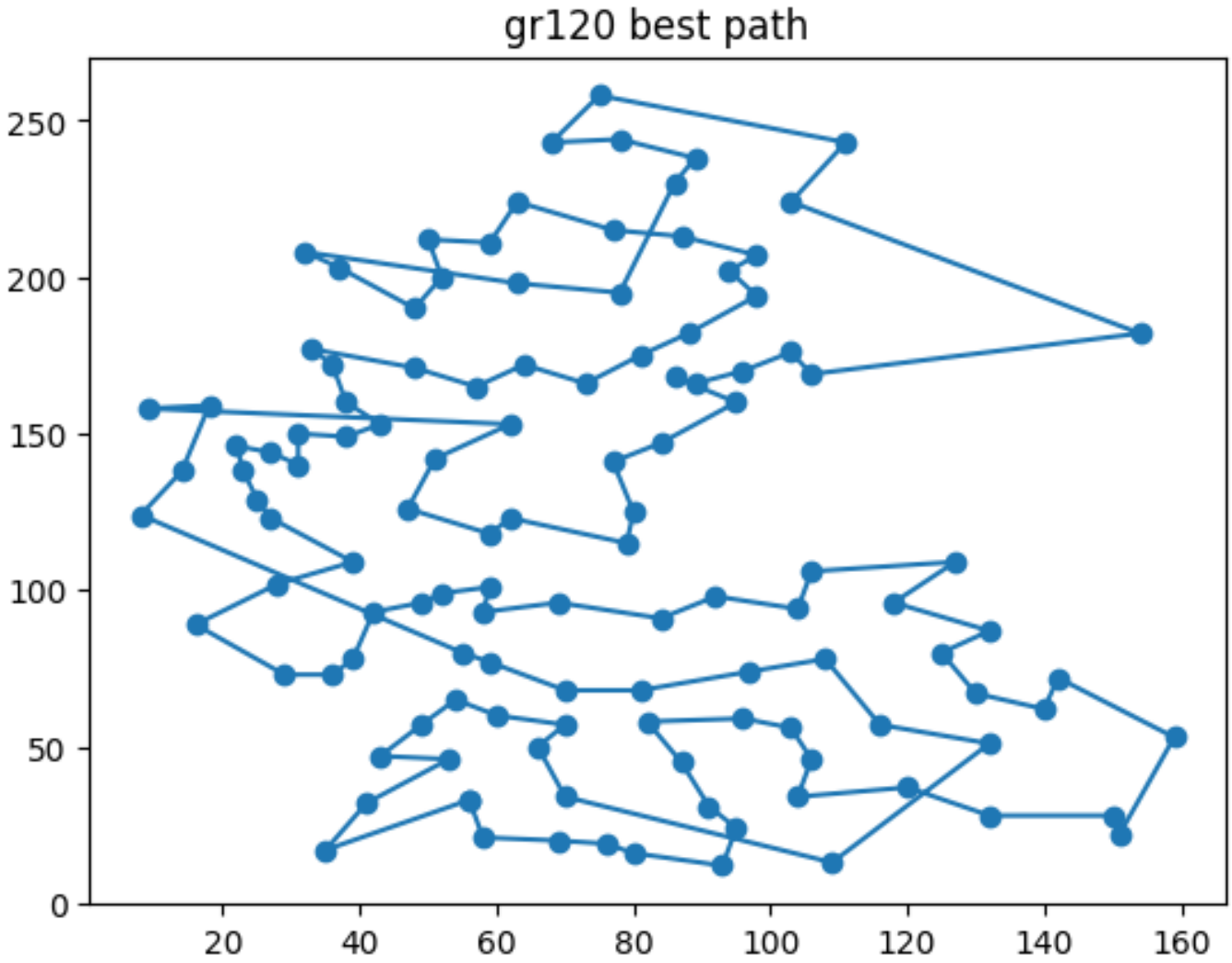


Fig. 7: gr120's best route from among 10 iterations